

T_i 要对 R_i 中某个元组加 S 锁,则要首先对关系 R_i 和数据库加 IS 锁。

② **IX 锁**。如果对一个数据对象加 IX 锁,表示它的后裔结点拟(意向)加 X 锁。例如,事务 T_i 要对 R_i 中某个元组加 X 锁,则要首先对关系 R_i 和数据库加 IX 锁。

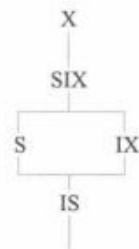
③ **SIX 锁**。如果对一个数据对象加 SIX 锁,表示对它加 S 锁,再加 IX 锁,即 $SIX = S + IX$ 。例如对某个表加 SIX 锁,则表示该事务要读整个表(所以要对该表加 S 锁),同时会更新个别元组(所以要对该表加 IX 锁)。

图 12.12(a)给出了这些锁的相容矩阵,从中可以发现这 5 种锁的强度有如图 12.12(b)所示的偏序关系。所谓锁的强度是指它对其他锁的排斥程度。一个事务在申请封锁时以强锁代替弱锁是安全的,反之则不然。

T_i	T_j					
	S	X	IS	IX	SIX	—
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
—	Y	Y	Y	Y	Y	Y

注: Y=Yes,表示相容的请求;N=No,表示不相容的请求

(a) 封锁类型的相容矩阵



(b) 锁的强度偏序关系

图 12.12 加意向锁后锁的相容矩阵与偏序关系

在具有意向锁的多粒度封锁方法中,任意事务 T 要对一个数据对象加锁,必须先对它的上层结点加意向锁。申请封锁时应按自上而下的次序进行,释放封锁时则应按自下而上的次序进行。

例如,事务 T_i 要对关系 R_i 加 S 锁,则要首先对数据库加 IS 锁。检查数据库是否已加了不相容的锁(X 锁),然后检查 R_i 是否已加了不相容的锁(X 或 IX 锁),不再需要搜索和检查 R_i 中的各元组是否加了不相容的锁(X 锁)。

具有意向锁的多粒度封锁方法提高了系统的并发度,减少了加锁和解锁的开销,已在实际的数据库管理系统产品中得到广泛应用。

*12.9 其他并发控制机制

并发控制的方法除了封锁技术外还有时间戳方法、乐观方法和多版本并发控制方法等。这里做一个概要的介绍。

时间戳方法给每一个事务“盖”上一个时标,即事务开始执行的时间。每个事务具有唯一的时间戳,并按照这个时间戳来解决事务的冲突操作。如果发生冲突操作,就回滚具有较早时

间截的事务，以保证其他事务的正常执行，被回滚的事务被赋予新的时间戳并从头开始执行。

乐观方法认为事务执行时很少发生冲突，因此不对事务进行特殊的管制，而是让它自由执行，事务提交前再进行正确性检查。如果检查后发现该事务执行中出现过冲突并影响了可串行性，则拒绝提交并回滚该事务。乐观方法又被称为验证方法。

多版本并发控制(MVCC)方法是指在数据库中通过维护数据对象的多个版本信息来实现高效并发控制的一种策略。

12.9.1 多版本并发控制方法

版本(version)是指数据库中数据对象的一个快照，记录了数据对象某个时刻的状态。随着计算机系统存储设备价格的不断降低，可以考虑为数据库系统的数据对象保留多个版本，以提高系统的并发操作程度。例如，有一个数据对象 $A=5$ ，有两个事务 T_1 、 T_2 ，其中 T_1 是写事务， T_2 是读事务。假定先启动 T_1 ，后启动 T_2 。按照传统的封锁协议， T_2 必须等待 T_1 执行结束释放 A 上的封锁后才能获得对 A 的封锁。也就是说， T_1 和 T_2 实际上是串行执行的，如图 12.13(a) 所示。如果在 T_1 准备写 A 时不是等待，而是为 A 生成一个新的版本(表示为 A')，那么 T_2 就可以继续在 A' 上执行。只是在 T_2 准备提交时要检查 T_1 是否已经完成。如果 T_1 已经完成了， T_2 就可以放心地提交；如果 T_1 还没有完成，那么 T_2 必须等待直到 T_1 完成。这样既能保持事务执行的可串行性，又提高了事务执行的并行度。这就是多版本并发控制方法的原理，如图 12.13(b) 所示。



图 12.13 封锁方法与多版本并发控制方法示意图

在多版本并发控制方法中，每个 $\text{write}(Q)$ 操作都创建 Q 的一个新版本，这样一个数据对象就有一个版本序列 $Q_1, Q_2, \dots, Q_m$ 与之相关联。每一个版本 Q_k 拥有版本的值、创建 Q_k 的事务的时间戳 $W\text{-timestamp}(Q_k)$ 和成功读取 Q_k 的事务的最大时间戳 $R\text{-timestamp}(Q_k)$ 。其中， $W\text{-tim-}$

$\text{estamp}(Q)$ 表示在数据项 Q 上成功执行 $\text{write}(Q)$ 操作的所有事务中的最大时间戳, $\text{R-timestamp}(Q)$ 表示在数据项 Q 上成功执行 $\text{read}(Q)$ 操作的所有事务中的最大时间戳。

用 $\text{TS}(T)$ 表示事务 T 的时间戳, $\text{TS}(T_i) < \text{TS}(T_j)$ 表示事务 T_i 在事务 T_j 之前开始执行。MVCC 协议描述如下:

假设版本 Q_k 具有小于或等于 $\text{TS}(T)$ 的最大时间戳。

若事务 T 发出 $\text{read}(Q)$, 则返回版本 Q_k 的内容。

若事务 T 发出 $\text{write}(Q)$, 则:

当 $\text{TS}(T) < \text{R-timestamp}(Q_k)$ 时, 回滚 T ;

当 $\text{TS}(T) = \text{W-timestamp}(Q_k)$ 时, 覆盖 Q_k 的内容。

否则, 创建 Q 的新版本。

若一个数据对象的两个版本 Q_k 和 Q_l , 其 W-timestamp 都小于系统中时间最长的事务的时间戳, 那么这两个版本中较旧的那个版本将不再被用到, 因而可以从系统中删除。

多版本并发控制方法利用物理存储上的多版本来维护数据的一致性。这就意味着当检索数据库时, 每个事务都能看到一个数据的一段时间前的快照, 而不管正在处理的数据当前的状态。多版本并发控制方法和封锁方法相比, 主要的好处是消除了数据库中数据对象读和写操作的冲突, 有效地提高了系统的性能。

多版本并发控制方法有利于提高事务的并发度, 但也会产生大量的无效版本; 而且在事务结束时刻, 其所影响的元组的有效性不能马上确定, 这就为保存事务执行过程中的状态提出了难题。这些都是实现多版本并发控制方法的一些关键技术。

12.9.2 改进的多版本并发控制方法

多版本并发控制方法可以进一步改进。区分事务的类型为只读事务和更新事务。对于只读事务, 发生冲突的可能性很小, 可以采用多版本时间戳。对于更新事务, 采用较保守的两段锁(2PL)协议。这样的混合协议称为 MV2PL。具体做法如下。

除了传统的共享型锁(S锁)和排他型锁(X锁)外, 引进一个新的封锁类型, 称为验证锁(certify-lock, 简称C锁)。验证锁的相容矩阵如图 12.14 所示。

T_1	T_2		
	S	X	C
S	Y	Y	N
X	Y	N	N
C	N	N	N

注: Y=Yes, 表示相容的请求; N=No, 表示不相容的请求

图 12.14 验证锁的相容矩阵

注意：在这个相容矩阵中，S 锁和 X 锁变得是相容的了。这样当某个事务写数据对象时，允许其他事务读数据（当然，写操作将生成一个新的版本，而读操作就是在旧的版本上读）。一旦写事务要提交，必须首先获得在那些加了 X 锁的数据对象上的 C 锁。由于 C 锁和 S 锁是不相容的，所以为了得到 C 锁，写事务不得不延迟提交，直到所有被它加上 X 锁的数据对象都被所有那些正在读它们的事务释放。一旦写事务获得 C 锁，系统就可以丢弃数据对象的旧值，代之以新版本，然后释放 C 锁，提交事务。

在这里，系统最多只要维护数据对象的两个版本，多个读操作可以和一个写操作并发地执行。这种情况是传统的两段锁所不允许的，从而提高了读写事务之间的并发度。

目前很多商用数据库管理系统，如 Oracle、Kingbase ES 都采用的是 MV2PL 协议。

MV2PL 协议把封锁机制和时间戳方法相结合，维护一个数据的多个版本，即对于关系表上的每一个写操作产生一个新版本，同时会保存前一次修改的数据版本。MV2PL 协议和封锁机制相比，主要的好处是在多版本并发控制中对读数据的锁要求与写数据的锁要求不冲突，所以读不会阻塞写，而写也从不阻塞读，从而使读写操作没有冲突，有效地提高了系统并发性。

现在许多数据库产品都使用了多版本并发控制技术，但是这些产品的实现细节各不相同，有兴趣的读者可参考相关文献。

本章小结

数据库的重要特征是能为多个用户提供数据共享。数据库管理系统必须提供并发控制机制来协调并发用户的并发操作，以保证并发事务的隔离性和一致性，确保数据库的一致性。

数据库的并发控制以事务为单位，通常使用封锁技术实现并发控制。本章介绍了最常用的封锁方法和三级封锁协议。不同的封锁和不同级别的封锁协议所提供的系统一致性保证是不同的。对数据对象施加封锁会带来活锁和死锁问题，数据库一般采用先来先服务、死锁诊断和解除等技术来预防活锁和死锁的发生。并发控制机制调度并发事务操作是否正确的判别准则是可串行性，两段锁是可串行化调度的充分条件，但不是必要条件。因此，两段锁可以保证并发事务调度的正确性。

不同的数据库管理系统提供的封锁类型、封锁协议、达到的数据一致性级别不尽相同，但是其依据的基本原理和技术是共同的。作为选读内容，本章还简要介绍了时间戳方法、乐观方法和多版本并发控制方法等其他并发控制方法。

习 题 12

1. 在数据库中为什么要采用并发控制？并发控制技术能保证事务的哪些特性？
2. 并发操作可能会产生哪几类数据不一致？用什么方法能避免各种不一致的情况？
3. 事务的隔离级别都有哪些，事务隔离级别与数据一致性的关系是什么？

4. 什么是封锁？基本的封锁类型有几种？试述它们的含义。
5. 如何用封锁机制保证数据的一致性？
6. 什么是活锁？试述活锁的产生原因和解决方法。
7. 什么是死锁？请给出预防死锁的若干方法。
8. 请给出检测死锁发生的一种方法。当发生死锁后如何解除死锁？
9. 什么样的并发调度是正确的调度？
10. 设 T_1 、 T_2 、 T_3 是如下的三个事务 (A 的初值为 0)。
 $T_1: A := A + 2;$
 $T_2: A := A * 2;$
 $T_3: A := A ** 2; \text{ (即 } A \leftarrow A^2 \text{)}$
 - ① 若这三个事务允许并发执行，则有多少种可能的正确结果？请一一列举出来。
 - ② 请给出一个可串行化的调度，并给出执行结果。
 - ③ 请给出一个非串行化的调度，并给出执行结果。
 - ④ 若这三个事务都遵守两段锁协议，请给出一个不产生死锁的可串行化调度。
 - ⑤ 若这三个事务都遵守两段锁协议，请给出一个产生死锁的调度。
11. 今有三个事务的一个调度 $R_3(B)R_1(A)W_3(B)R_2(B)R_2(A)W_2(B)R_1(B)W_1(A)$ ，该调度是冲突可串行化的调度吗？为什么？
 12. 试证明若并发事务遵守两段锁协议，则对这些事务的并发调度是可串行化的。
 13. 举例说明对并发事务的一个调度是可串行化的，而这些并发事务不一定遵守两段锁协议。
 14. 考虑如下的调度，说明这些调度集合之间的包含关系。
 - ① 正确的调度。
 - ② 可串行化的调度。
 - ③ 遵循两段锁的调度。
 - ④ 串行调度。
15. 考虑如下的 T_1 和 T_2 两个事务。
 $T_1: R(A); R(B); B = A + B; W(B)$
 $T_2: R(B); R(A); A = A + B; W(A)$
 - ① 改写 T_1 和 T_2 ，增加加锁操作和解锁操作，并要求遵循两段锁协议。
 - ② 说明 T_1 和 T_2 的执行是否会引起死锁，给出 T_1 和 T_2 的一个调度并说明之。
16. 为什么要引进意向锁？意向锁的含义是什么？
17. 试述常用的意向锁 (IS 锁、IX 锁、SIX 锁)，给出这些锁的相容矩阵。

第12章实验 并发控制

并发控制实验。掌握数据库并发控制的基本原理及应用方法。验证并发操作带来的数据不一致性问题，包括丢失修改、不可重复读和脏读等情况。要求通过取消查询分析器的自动提交功能，创建两个不同的用户，分别登录查询分析器，同时打开两个客户端；通过使用 SQL 语句设计具体例子，展示不同封锁级别的应用场景，验证各种封锁级别的并发控制效果，以进一步理解封锁技术如何解决事务并发导致的问题。

参考文献 12

- [1] BERNSTEIN P A, HADZILACOS V, GOODMAN N. Concurrency control and recovery in data base systems [M]. Addison Wesley, 1988.
- [2] ESWARAN K P, GRAY J N, LORIE R A, et al. The notions of consistency and predicate locks in a data base system[J]. Communications of the ACM, 1976, 19(11): 624-633.
- 文献[2]给出了可串行性概念的形式化定义。
- [3] GRAY J N, LORIE R A, PUTZOLU G R. Granularity of locks in a large shared data base[C]. Proceedings of the 1st International Conference on Very Large Data Bases, 1975: 428-451.
- 文献[3]综述了多粒度封锁协议。
- [4] GRAY J, REUTER A. Transaction processing: concepts and techniques[M]. San Francisco: Morgan Kaufmann, 1992.
- [5] PAPADIMITRIOU C H. The Serializability of concurrent database updates[J]. Journal of the ACM, 1979, 26(4): 631-653.
- 文献[5]给出了与可串行性相关的研究结果。
- [6] KEDEM Z M, SILBERSCHATZ A. Locking protocols: from exclusive to shared locks[J]. Journal of the ACM, 1983, 30(4): 787-804.
- 文献[6]讨论了封锁协议。
- [7] BAYER R, SCHKOLNICK M. Concurrency of operating on B-trees[J]. Acta Informatica, 1977, 9: 1.
- 文献[7]提出了对B树进行并发存取的算法。
- [8] LEHMAN P L, YAO S B. Efficient locking for concurrent operations on B-trees[J]. ACM Transactions on Database Systems, 1981, 6(4): 650-670.
- [9] BERNSTEIN P A, GOODMAN N. Timestamp-based algorithms for concurrency control in distributed database systems[C]. Proceedings of the 6th International Conference on Very Large Data Bases, 1980, 6: 285-300.
- 文献[9]讨论了各种基于时间戳的并发控制算法。
- [10] BERNSTEIN P A, GOODMAN N. Timestamp-based algorithms for concurrency control in distributed database systems[C]. Proceedings of the 6th International Conference on Very Large Databases, 1980: 285-300.
- [11] KUNG H T, ROBINSON J T. On optimistic methods for concurrency control[J]. ACM Transaction on Database Systems, 1981, 6(2): 213-226.
- [12] SILBERSCHATZ A. A multi-version concurrency scheme with no rollbacks[C]. Proceedings of the 1st ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing, 1982: 216-223.
- [13] BERNSTEIN P A, GOODMAN N. Multi-version concurrency Control-Theory and algorithms[J]. ACM Transaction on Database Systems, 1983, 8(4): 465-483.
- [14] CAREY M J. Multiple versions and the performance of optimistic concurrency control[R]. Computer Sciences Technical Report 517, 1983.
- [15] FALEIRO J M, ABADI D J. Rethinking serializable multiversion concurrency control[C]. Proceedings of the VLDB Endowment, 2015, 8(11): 1190-1201.
- 文献[15]提出了一种新的多版本数据库并发控制协议。

本章知识点讲解微视频：



数据异常与隔离
级别



封锁与封锁协议



可串行化与冲突
可串行化



两段锁协议

* 第 13 章 数据库管理系统概述

本章导读

数据库管理系统是对数据库中的共享数据进行有效的组织、存储、管理和存取的软件系统。本章主要阐述数据库管理系统的基本功能、系统结构及主要实现技术。

本章首先介绍数据库管理系统的基本功能，然后讲解数据库管理系统的语言处理层、数据存取层和数据存储层中缓冲区的层次结构，以及这三层结构是如何配合完成数据库查询处理请求的。最后介绍数据库的物理组织。

本章不是针对数据库管理系统的设计人员编写的，而是面向数据库管理员和数据库应用系统开发人员的，目的是使他们从宏观和总体的角度掌握数据库管理系统的基本概念和基本原理，以便更好地使用和维护数据库管理系统。

13.1 数据库管理系统的基本功能

数据库管理系统已经发展成为继操作系统之后最复杂的系统软件。前面已讲过，数据库管理系统主要是实现对共享数据有效的组织、存储、管理和存取。围绕数据，数据库管理系统应具有如下基本功能。

① **数据库定义和创建。**创建数据库主要是用数据定义语言定义和创建数据库模式、外模式、内模式等数据库对象。在关系数据库中就是建立数据库(或模式)、表、视图、索引等，还有创建用户、定义安全保密(如用户口令、级别、角色、存取权限)、定义数据库的完整性约束。这些定义存储在数据字典(亦称为系统目录)中，是数据库管理系统运行的基本依据。

② **数据组织、存储和管理。**数据库管理系统要分类组织、存储和管理各种数据，包括数据字典、用户数据、存取路径等；要确定以何种文件结构和存取方式在存储器上组织这些数据，以及如何实现数据之间的联系。数据组织和存储的基本目标是提高存储空间利用率和方便存取，提供多种存取方法(如索引查找、哈希查找、顺序查找等)以提高存取效率。

③ **数据存取。**数据库管理系统提供用户对数据的操作功能，实现对数据库数据的检索、